

Group 9 Assignment Report

An Evolutionary Algorithm for the Travelling Salesman Problem

Christos Zampounis Mehmet Cihan Yalcin Péter Dávid Hajdu
Pierre Kasparian Prajval Parthiban

September 30, 2026

1 Introduction

The goal of this assignment is to design, implement and experimentally evaluate an evolutionary algorithm for a combinatorial optimisation problem. We chose the Travelling Salesman Problem, whose discrete, permutation-based search space and relevance to routing and logistics make it a natural and challenging testbed for evolutionary computation. This report presents our problem formulation, the evolutionary approach and operators we developed, and an experimental analysis of its solution quality and convergence behaviour.

2 The Travelling Salesman Problem

2.1 Mathematical Formulation

The Travelling Salesman Problem (TSP) is a classical combinatorial optimisation problem that, despite its simple formulation, is difficult to solve: given a set of N cities and the distances between them, the task is to find the shortest closed tour that visits every city exactly once. The problem is NP-hard and no exact algorithm can solve it in polynomial time. Following the formulation of Potvin [1], we model the problem on a complete weighted graph $G = (V, E)$, where $V = \{1, \dots, N\}$ is the set of cities and d_{ij} is the distance between cities i and j . The decision variables are $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ if and only if the tour travels directly from city i to city j . The objective is to minimise the total travelled distance,

$$\text{Minimise} \quad \sum_{i=1}^N \sum_{j=1}^N d_{ij} x_{ij}, \quad (1)$$

subject to the degree constraints

$$\sum_{j=1}^N x_{ij} = 1, \quad i = 1, \dots, N, \quad \sum_{i=1}^N x_{ij} = 1, \quad j = 1, \dots, N, \quad (2)$$

which require every city to have exactly one incoming and one outgoing edge, and the subtour-elimination constraints

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad \forall S \subset V, \quad 2 \leq |S| \leq N - 1, \quad (3)$$

which forbid partial cycles over any proper subset S of the cities. Without constraints (3), the model reduces to the assignment problem and its optimal solution generically consists of several disconnected subtours rather than a single tour. For symmetric instances, such as

the Euclidean TSP (ETSP) considered in this report, the optimal solution of the assignment-problem relaxation typically contains many two-vertex subtours; such instances are therefore better tackled by specialised algorithms that exploit this structure [1].

Because an evolutionary algorithm operates on orderings rather than on binary edge variables, it is convenient to state the equivalent tour-as-permutation formulation. A candidate solution is a permutation $\pi = (\pi_1, \dots, \pi_N)$ of the cities, and its cost is the length of the cycle that visits them in that order,

$$C(\pi) = \sum_{k=1}^{N-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_N, \pi_1}. \quad (4)$$

The evolutionary algorithm therefore seeks a permutation minimising Equation (4); this representation is the one adopted throughout this report.

3 Algorithm

Each individual is represented as a permutation of all cities, where the order of the cities defines the tour and the final city is connected back to the first. The initial population consists of p randomly generated permutations. The cost of an individual is given by Equation (4). Since the TSP is a minimisation problem, this cost is converted into a maximisation-based fitness function defined as

$$F(\pi) = \frac{1}{C(\pi)}. \quad (5)$$

Consequently, shorter tours receive higher fitness values and are favoured during selection.

In each generation, a fraction $(1 - r)$ of the population is selected and carried over unchanged. The remaining fraction r is replaced by offspring generated from selected parent pairs. Tournament selection is used to select parents. For each parent a small number of individuals are sampled randomly from the population, and the individual with the highest fitness is selected. This process is repeated to obtain the second parent. Tournament selection was chosen because it is simple to implement and provides selection pressure towards shorter tours while maintaining population diversity.

Order Crossover (OX) is used to generate offspring because it preserves valid permutations. A segment from one parent is copied to the offspring, while the remaining cities are inserted according to their order in the second parent, skipping already used cities. Two offspring are generated for each parent pair by swapping the parent roles.

After crossover, swap mutation is applied to a fraction m of the new population. Two randomly selected positions in an individual are exchanged, which introduces variation while preserving a valid permutation. The process is repeated for G generations, and the best tour found during the run is returned.

Algorithm 1 Genetic Algorithm for the Travelling Salesman Problem

Require: cities, population size p , replacement rate r , mutation rate m , number of generations G

Ensure: best tour found

```

 $P \leftarrow p$  random permutations of the cities
evaluate the fitness of each individual in  $P$ 
for  $g = 1$  to  $G$  do
     $P_s \leftarrow$  the best  $(1 - r)p$  individuals from  $P$ 
    for  $i = 1$  to  $(rp)/2$  do
        select parents  $h_1, h_2$  from  $P$  using tournament selection
         $o_1 \leftarrow \text{ORDERCROSSOVER}(h_1, h_2)$ 
         $o_2 \leftarrow \text{ORDERCROSSOVER}(h_2, h_1)$ 
        add  $o_1$  and  $o_2$  to  $P_s$ 
    end for
    apply SWAPMUTATION to a fraction  $m$  of  $P_s$ 
     $P \leftarrow P_s$ 
    evaluate the fitness of each individual in  $P$ 
    update the best tour found
end for
return best tour

```

4 Experimental part

5 Results and Analysis

6 Conclusions

References

- [1] J.-Y. Potvin, “Genetic algorithms for the traveling salesman problem,” *Annals of Operations Research*, vol. 63, no. 3, pp. 339–370, 1996. <https://doi.org/10.1007/BF02125403>.